

LESSON 303.6 · JAVA FUNDAMENTALS

Arrays: Storing and Processing Data

Where `scans.length` comes from, and everything you can do to a shelf of packages once you have one.

Separate variables stop working at about three

Last lesson's loop needed a batch to loop over. This is what a batch looks like without arrays.

RUNNABLE JAVA PROGRAM

```
public class SeparateVariables {  
    public static void main(String[] args) {  
        double weight1 = 8.0;  
        double weight2 = 25.0;  
        double weight3 = 0.5;  
  
        double total = weight1 + weight2 + weight3;  
        System.out.println("Total: " + total + " lbs");  
    }  
}
```

There is no loop you can write over `weight1`, `weight2` and `weight3`. Three names are three separate variables, and a loop needs one name it can move through.

An array gives a batch a single name and a position for every item.

An array is a fixed run of numbered slots, all the same type

Java reserves the whole run at once, side by side, and numbers the slots from zero. That is why reading position 4 is instant.



The name `shelf` refers to the whole run. `shelf[2]` means "the slot two steps from the start" — which is why the first slot is `shelf[0]` and not `shelf[1]`. Zero is not a quirk; it is a distance.

Reading `shelf[2]` never searches. Java computes where slot 2 is and goes straight there.

THE THREE FACTS

Three facts explain every array bug you will hit this lesson

Keep these three sentences. When an array surprises you, one of them is the reason, and knowing which one tells you where to look.

FIXED LENGTH	The size is chosen at creation and never changes. There is no add and no remove — only overwrite, or build a bigger array.
A REFERENCE	The variable holds the <i>address</i> of the run, not the data. Two variables can point at the same run, and often do by accident.
ZERO-BASED	Slots are numbered 0 to $\text{length} - 1$. The last valid index is always one less than the length.

Out of bounds is fact three. An unexpected change is fact two. "I cannot add an item" is fact one.

There are three ways to create one, and they differ in what you know

Use the form that matches what you have: a size, the values, or both.

RUNNABLE JAVA PROGRAM

```
public class CreatingArrays {  
    public static void main(String[] args) {  
        double[] declared = new double[3];  
        double[] listed = {8.0, 25.0, 0.5};  
        double[] both = new double[]{8.0, 25.0, 0.5};  
  
        System.out.println("declared[0] = " + declared[0]);  
        System.out.println("listed[0] = " + listed[0]);  
        System.out.println("both[2] = " + both[2]);  
    }  
}
```

OUTPUT

```
declared[0] = 0.0  
listed[0] = 8.0  
both[2] = 0.5
```

The short `{8.0, 25.0}` form only works on the line that declares the variable. Anywhere else you need `new double[]{...}`.

A new array is already full — of the type's default value

`new double[3]` does not give you an empty array. It gives you three zeros, and that difference shows up in your totals.

RUNNABLE JAVA PROGRAM

```
public class DefaultValues {  
    public static void main(String[] args) {  
        int[] counts = new int[3];  
        double[] weights = new double[3];  
        boolean[] flags = new boolean[3];  
        String[] labels = new String[3];  
  
        System.out.println("int slot: " + counts[0]);  
        System.out.println("double slot: " + weights[0]);  
        System.out.println("boolean slot: " + flags[0]);  
        System.out.println("String slot: " + labels[0]);  
    }  
}
```

OUTPUT

```
int slot: 0  
double slot: 0.0  
boolean slot: false  
String slot: null
```

An unfilled `String` slot is `null`, and calling a method on it is the `NullPointerException` you will meet in Lesson 303.10.

length is how many slots exist; the last index is one less

This single off-by-one is the most common array mistake in the language, and the fix is a habit, not a rule.

RUNNABLE JAVA PROGRAM

```
public class ShelfBounds {  
    public static void main(String[] args) {  
        double[] shelf = {8.0, 25.0, 0.5, 12.0};  
  
        System.out.println("length: " + shelf.length);  
        System.out.println("first slot: " + shelf[0]);  
        System.out.println("last slot: " + shelf[shelf.length - 1]);  
    }  
}
```

OUTPUT

```
length: 4  
first slot: 8.0  
last slot: 12.0
```

Never type the size as a number. Write `shelf.length`, and the loop keeps working when the data changes size.

Reaching past the last slot throws, it does not return nothing

Java checks every index at run time. A bad index stops the program at that line rather than handing you a wrong answer.

BROKEN PROGRAM

```
public class OutOfBounds {  
    public static void main(String[] args) {  
        double[] shelf = {8.0, 25.0, 0.5, 12.0};  
        System.out.println(shelf[4]);  
    }  
}
```

WHAT YOU GET

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:  
Index 4 out of bounds for length 4  
at OutOfBounds.main(OutOfBounds.java:4)
```

Read the message literally: it tells you the index you asked for and the length you had. Almost always the fix is $<$ instead of $\leq$.

Fill the first and last slot

Three blanks. Every one of them is fact three — slots are numbered from zero.

REQUIREMENTS

- The shelf holds four packages.
- Put 2.5 in the very first slot.
- Put 9.0 in the very last slot, without typing the number 3.

Expected output

First: 2.5

Last: 9.0

If you get an `ArrayIndexOutOfBoundsException`, the third blank is
- 0 when it needs to be something else.

COMPLETE THE PLAN

```
public class SlotPractice {  
    public static void main(String[] args) {  
        double[] shelf = new double[____];  
  
        shelf[____] = 2.5;  
        shelf[shelf.length ____] = 9.0;  
  
        System.out.println("First: " + shelf[0]);  
        System.out.println("Last: " + shelf[shelf.length - 1]);  
    }  
}
```

Four slots, numbered 0 to 3 — so the last index is length minus one

Writing `shelf.length - 1` instead of 3 is the whole point of the exercise.

RUNNABLE SOLUTION (BODY)

```
double[] shelf = new double[4];  
  
shelf[0] = 2.5;  
shelf[shelf.length - 1] = 9.0;
```

- NEW DOUBLE[4]** "holds four packages" — the length is the count, not the last index
- SHELF[0]** "the very first slot" — zero is a distance from the start, so the first is zero away
- SHELF.LENGTH - 1** "without typing the number 3" — this line still works if the shelf grows to 40

Hard-coding 3 is correct today and a bug the first time someone changes the size.

SECTION TWO

Walking an array

Last lesson's control plan, now with a real batch under it.

The indexed for loop and the array were built for each other

Start at 0, continue while `i < length`, progress by one. Memorise this header; you will write it for the rest of your career.

RUNNABLE JAVA PROGRAM

```
public class WalkTheShelf {  
    public static void main(String[] args) {  
        double[] shelf = {8.0, 25.0, 0.5, 12.0};  
  
        for (int i = 0; i < shelf.length; i++) {  
            System.out.println("slot " + i + ": " + shelf[i] + " lbs");  
        }  
    }  
}
```

OUTPUT

```
slot 0: 8.0 lbs  
slot 1: 25.0 lbs  
slot 2: 0.5 lbs  
slot 3: 12.0 lbs
```

Zero pairs with `<`. That pairing is what makes the loop visit every slot and no invalid ones.

The enhanced for loop finally has something to be enhanced over

You met it last lesson without an array to point it at. This is what it was built for — and what it cannot do.

Indexed — you own the start, the boundary and the step

```
for (int i = 0; i < shelf.length; i++) {  
    System.out.println(shelf[i]);  
}
```

Enhanced — Java owns all three; you get the value

```
for (double weight : shelf) {  
    System.out.println(weight);  
}
```

Read the colon as *in*. There is no index to declare, no boundary to get wrong, and no way to run off the end — which removes most of the bugs in this deck at the cost of the control the next slide needs.

Use enhanced for when you need values. Use indexed for when you need positions.

An enhanced for loop cannot change the array it is walking

The loop variable is a fresh copy of the slot's value. Assigning to it changes the copy and the array never hears about it.

BROKEN PROGRAM

```
public class ForEachCannotModify {  
    public static void main(String[] args) {  
        double[] shelf = {8.0, 25.0, 0.5};  
  
        for (double weight : shelf) {  
            weight = weight * 2;  
        }  
  
        System.out.println("first slot is still " + shelf[0]);  
    }  
}
```

OUTPUT

```
first slot is still 8.0
```

To write into a slot you must name the slot: `shelf[i] = shelf[i] * 2;` inside an indexed loop.

Printing an array prints its address, not its contents

This is fact two showing itself the first time. The variable holds a reference, so that is what gets printed.

RUNNABLE JAVA PROGRAM

```
import java.util.Arrays;

public class PrintingAnArray {
    public static void main(String[] args) {
        double[] shelf = {8.0, 25.0, 0.5};

        System.out.println(shelf);
        System.out.println(Arrays.toString(shelf));
    }
}
```

OUTPUT

```
[D@659e0bfd
[8.0, 25.0, 0.5]
```

[D means "array of double" and the hex changes every run. Reach for `Arrays.toString` whenever you are debugging.

Swap two slots

Swap what is in slot 1 with what is in slot 3. Three blanks, and the order of the three lines is what makes it work.

REQUIREMENTS

- Slot 1 must end up holding what slot 3 held.
- Slot 3 must end up holding what slot 1 held.
- Neither value may be lost along the way.

Expected output

[A, D, C, B]

Getting [A, D, C, D] means you overwrote slot 1 before you saved what was in it. That is why the first line exists.

COMPLETE THE PLAN

```
import java.util.Arrays;

public class SwapSlots {
    public static void main(String[] args) {
        String[] bays = {"A", "B", "C", "D"};

        String held = bays[___];
        bays[1] = bays[___];
        bays[3] = ___;

        System.out.println(Arrays.toString(bays));
    }
}
```


A swap needs a third variable because assignment destroys

`bays[1] = bays[3]` does not exchange anything. It overwrites, and whatever was in slot 1 is gone.

RUNNABLE SOLUTION (BODY)

```
String held = bays[1];  
bays[1] = bays[3];  
bays[3] = held;
```

HELD = BAYS[1]	"neither value may be lost" — save the one that is about to be overwritten
BAYS[1] = BAYS[3]	"slot 1 holds what slot 3 held" — safe now, because slot 1's old value is parked
BAYS[3] = HELD	"slot 3 holds what slot 1 held" — the parked value goes back in
RESULT	[A, D, C, B] — slots 0 and 2 were never touched

You will write this exact three-line dance again in every sort algorithm you ever read.

SECTION THREE

An array variable is a reference

Fact two, in full. This section is where the surprising bugs live.

Assigning an array does not copy it — it makes a second name

Both variables now refer to the same run of slots, so a write through either one is visible through the other.

BROKEN PROGRAM

```
import java.util.Arrays;

public class ReferenceAlias {
    public static void main(String[] args) {
        double[] original = {8.0, 25.0, 0.5};
        double[] second = original;

        second[0] = 99.0;

        System.out.println("original: " + Arrays.toString(original));
        System.out.println("second: " + Arrays.toString(second));
    }
}
```

OUTPUT

```
original: [99.0, 25.0, 0.5]
second: [99.0, 25.0, 0.5]
```

Nobody wrote to `original`, and `original` changed. That is fact two.

Passing an array to a method hands over the reference

The method cannot replace your array, but it can rewrite every slot in it — and the caller sees the result.

RUNNABLE JAVA PROGRAM

```
import java.util.Arrays;

public class PassAnArray {
    public static void main(String[] args) {
        double[] shelf = {8.0, 25.0, 0.5};

        zeroOutFirst(shelf);

        System.out.println("back in main: " + Arrays.toString(shelf));
    }

    public static void zeroOutFirst(double[] slots) {
        slots[0] = 0.0;
    }
}
```

OUTPUT

```
back in main: [0.0, 25.0, 0.5]
```

A method taking an array can change your data without returning anything. Read its name carefully before you call it.

== compares references, so two identical arrays are not equal

== asks "is this the same run of slots?" It never asks about the contents.

RUNNABLE JAVA PROGRAM

```
import java.util.Arrays;

public class CompareArrays {
    public static void main(String[] args) {
        double[] left = {8.0, 25.0, 0.5};
        double[] right = {8.0, 25.0, 0.5};

        System.out.println("left == right: " + (left == right));
        System.out.println("Arrays.equals: " + Arrays.equals(left, right));
    }
}
```

OUTPUT

```
left == right: false
Arrays.equals: true
```

Same question as == versus .equals() on Strings, and the same answer: == is about identity.

Arrays.copyOf makes a real second array

Now a write to the copy leaves the original alone — which is exactly what you need before you sort something you must not reorder.

RUNNABLE JAVA PROGRAM

```
import java.util.Arrays;

public class CopyAnArray {
    public static void main(String[] args) {
        double[] original = {8.0, 25.0, 0.5};
        double[] copy = Arrays.copyOf(original, original.length);

        copy[0] = 99.0;

        System.out.println("original: " + Arrays.toString(original));
        System.out.println("copy: " + Arrays.toString(copy));
    }
}
```

OUTPUT

```
original: [8.0, 25.0, 0.5]
copy: [99.0, 25.0, 0.5]
```

The second argument is the new length, so `copyOf` is also how you "grow" an array: copy it into a bigger one.

KNOWLEDGE CHECK

Which of the three facts is this question about?

Say the fact out loud before you pick an answer.

THINK IT THROUGH FIRST

WHAT DOES THIS PRINT?

```
int[] a = {1, 2, 3};  
int[] b = a;  
b[0] = 9;  
  
System.out.println(a[0]);
```

A. 1

B. 9

C. 0

D. It does not compile

b is a second name for the same run of slots

Fact two. `int[] b = a` copied a reference, not three integers.

ANSWER

WHAT DOES THIS PRINT?

```
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 9;

System.out.println(a[0]);
```

A. 1

B. 9

C. 0

D. It does not compile

WHY

Answer A (1) assumes the assignment made a copy. It did not: `a` and `b` refer to the same array, so writing through `b` is writing through `a`. To get 1 you would need `int[] b = Arrays.copyOf(a, a.length);`. This is the single most expensive array misunderstanding in real code, because nothing crashes — a number is just quietly wrong.

The three facts, and what each one costs you

Fixed length

no add, no remove — insert means shifting, and growing means copying

A reference

= aliases, == lies, methods can rewrite your data, copies are one level deep

Zero-based

the last index is length - 1, and <= in a loop header is a crash

Every remaining bug in this deck is one of these three wearing a different hat.

SECTION FOUR

The `java.util.Arrays` toolbox

Sorting, filling, searching and comparing — written once, by the JDK, so you do not write them again.

Six Arrays methods cover almost everything you will need

Add `import java.util.Arrays;` at the top of the file. Every method below is static, so you call it on the class.

CALL	WHAT IT DOES	WATCH OUT FOR
<code>Arrays.toString(a)</code>	Readable text of the contents	Only one level deep — use <code>deepToString</code> for 2D
<code>Arrays.sort(a)</code>	Sorts ascending, in place	It mutates the array you hand it
<code>Arrays.fill(a, v)</code>	Sets every slot to <code>v</code>	Overwrites everything, including data you wanted
<code>Arrays.copyOf(a, n)</code>	New array of length <code>n</code>	Longer than <code>a</code> means the extra slots get defaults
<code>Arrays.equals(a, b)</code>	Compares contents	One level deep — use <code>deepEquals</code> for 2D
<code>Arrays.binarySearch(a, v)</code>	Fast index lookup	Only valid on a sorted array

Two of these six mutate their argument. Knowing which two is most of using them safely.

SORTING

Arrays.sort rearranges the original, so sort a copy when order matters

Sorting is the clearest example of a method that changes your data without returning anything.

RUNNABLE JAVA PROGRAM

```
import java.util.Arrays;

public class SortACopy {
    public static void main(String[] args) {
        double[] weights = {8.0, 25.0, 0.5, 12.0, 3.0};

        double[] sorted = Arrays.copyOf(weights, weights.length);
        Arrays.sort(sorted);

        System.out.println("original: " + Arrays.toString(weights));
        System.out.println("sorted copy: " + Arrays.toString(sorted));
        System.out.println("heaviest: " + sorted[sorted.length - 1]);
    }
}
```

OUTPUT

```
original: [8.0, 25.0, 0.5, 12.0, 3.0]
sorted copy: [0.5, 3.0, 8.0, 12.0, 25.0]
heaviest: 25.0
```

Sorting a copy costs one line and preserves the arrival order the rest of your program may still need.

Sorting one of two parallel arrays silently corrupts your data

Two arrays that line up by position only stay lined up if you move them together. Sort one and the pairing is gone with no error.

BROKEN PROGRAM

```
import java.util.Arrays;

public class SortBreaksPairing {
    public static void main(String[] args) {
        String[] tracking = {"NP-1001", "NP-1002", "NP-1003"};
        double[] weights = {8.0, 25.0, 0.5};

        Arrays.sort(weights);

        System.out.println(Arrays.toString(tracking));
        System.out.println(Arrays.toString(weights));
        System.out.println("slot 0 now claims " + tracking[0]
            + " weighs " + weights[0]);
    }
}
```

OUTPUT

```
[NP-1001, NP-1002, NP-1003]
[0.5, 8.0, 25.0]
slot 0 now claims NP-1001 weighs 0.5
```

NP-1001 weighs 8.0. The program now bills it as 0.5 and nothing crashed. Lesson 303.7 fixes this properly with a class.

SECTION FIVE

Processing patterns

Aggregate, search, insert, delete, reverse — the five things anyone ever asks you to do to a list of data.

Seed a running best from the first element, never from zero

Total, largest and smallest are all the accumulator pattern. The smallest is where the seed value bites.

RUNNABLE JAVA PROGRAM

```
public class Aggregate {  
    public static void main(String[] args) {  
        double[] weights = {8.0, 25.0, 0.5, 12.0, 3.0};  
  
        double total = 0.0;  
        double heaviest = weights[0];  
        double lightest = weights[0];  
  
        for (int i = 0; i < weights.length; i++) {  
            total = total + weights[i];  
            if (weights[i] > heaviest) heaviest = weights[i];  
            if (weights[i] < lightest) lightest = weights[i];  
        }  
  
        System.out.println("Total: " + total + " lbs");  
        System.out.println("Heaviest: " + heaviest + " lbs");  
        System.out.println("Lightest: " + lightest + " lbs");  
    }  
}
```

OUTPUT

```
Total: 48.5 lbs  
Heaviest: 25.0 lbs  
Lightest: 0.5 lbs
```

Seed **lightest** at 0.0 and nothing is ever lighter, so the answer is always 0.0.

Linear search looks at everything until it finds a match

Works on any array in any order. Return the *index*, not the value, and use -1 to mean "not present".

RUNNABLE JAVA PROGRAM

```
public class FindTracking {  
    public static void main(String[] args) {  
        String[] tracking = {"NP-1001", "NP-1002", "NP-1003", "NP-1004"};  
        String wanted = "NP-1003";  
        int found = -1;  
  
        for (int i = 0; i < tracking.length; i++) {  
            if (tracking[i].equals(wanted)) {  
                found = i;  
                break;  
            }  
        }  
  
        if (found == -1) {  
            System.out.println(wanted + " is not on the shelf");  
        } else {  
            System.out.println(wanted + " is in slot " + found);  
        }  
    }  
}
```

OUTPUT

NP-1003 is in slot 2

.equals() for Strings, never == — same reference-versus-contents rule as arrays.

Binary search halves the problem, and demands a sorted array

On a million sorted items linear search reads up to a million slots; binary search reads about twenty.

RUNNABLE JAVA PROGRAM

```
import java.util.Arrays;

public class BinaryLookup {
    public static void main(String[] args) {
        int[] sortedIds = {1001, 1002, 1003, 1004, 1005};

        System.out.println(Arrays.binarySearch(sortedIds, 1004));
        System.out.println(Arrays.binarySearch(sortedIds, 1006));
    }
}
```

OUTPUT

```
3
-6
```

A negative result is not an error

It encodes where the value *would* go: the answer is - (insertion point) - 1. Here -6 means "not found, and it belongs at index 5."

The only test you need

A result of zero or more is a real index. A negative result means it is not there.

Sorting to enable one lookup is slower than just scanning. Sort when you will search many times.

Binary search on an unsorted array reports a wrong answer, not an error

It assumes order and never checks. Given disorder it walks confidently to the wrong half and reports nothing found.

BROKEN PROGRAM

```
import java.util.Arrays;

public class UnsortedLookup {
    public static void main(String[] args) {
        int[] ids = {1005, 1001, 1004, 1002, 1003};

        System.out.println(Arrays.binarySearch(ids, 1005));
    }
}
```

OUTPUT

-6

1005 is sitting in slot 0. The method reports -6 — "not present." No exception, no warning, just a confident lie.

Sort the array, or use a linear search. Never call `binarySearch` on data you did not sort yourself.

Find the lightest package and its slot

Three blanks. The first one is the seeding decision from two slides ago.

REQUIREMENTS

- Start the running minimum from the first package, not from zero.
- Compare each later package against the best so far.
- Remember the slot number, not just the weight.

Expected output

Lightest: 0.5 lbs at slot 2

Getting 0.0 means you seeded from zero. Getting slot 0 means the third blank updates the wrong variable.

COMPLETE THE PLAN

```
public class Lightest {  
    public static void main(String[] args) {  
        double[] weights = {8.0, 25.0, 0.5, 12.0, 3.0};  
  
        double lightest = weights[___];  
        int lightestSlot = 0;  
  
        for (int i = 1; i < weights.length; i++) {  
            if (weights[i] ___ lightest) {  
                lightest = weights[i];  
                lightestSlot = ___;  
            }  
        }  
  
        System.out.println("Lightest: " + lightest  
            + " lbs at slot " + lightestSlot);  
    }  
}
```

Seeding from `weights[0]` is why the loop can start at 1

The first element is not skipped — it is already the answer until something beats it.

RUNNABLE SOLUTION (BODY)

```
double lightest = weights[0];
int lightestSlot = 0;

for (int i = 1; i < weights.length; i++) {
    if (weights[i] < lightest) {
        lightest = weights[i];
        lightestSlot = i;
    }
}
```

- WEIGHTS[0]** "start from the first package" — a real value, so any real value can beat it
- < LIGHTEST** "compare against the best so far" — strictly less, so ties keep the earlier slot
- LIGHTESTSLOT = I** "remember the slot number" — both variables must update together, inside the same if

If the two updates ever get separated, you report one package's weight at another package's slot.

INSERT

Inserting means shifting, because the length cannot grow

Fact one. To make room at slot 2 you copy everything right by one — walking *backwards* so you do not overwrite as you go.

RUNNABLE JAVA PROGRAM

```
import java.util.Arrays;

public class InsertAtSlot {
    public static void main(String[] args) {
        String[] bays = {"A", "B", "D", "E", null};
        int used = 4;

        for (int i = used; i > 2; i--) {
            bays[i] = bays[i - 1];
        }
        bays[2] = "C";
        used++;

        System.out.println(Arrays.toString(bays));
        System.out.println("used slots: " + used);
    }
}
```

OUTPUT

```
[A, B, C, D, E]
used slots: 5
```

Walk forwards and you smear the first value across the rest. Direction is the whole trick.

DELETE

Deleting leaves a stale value behind unless you clear it

Shifting left covers the hole, but the last slot still holds the old final value. Only your used counter knows it is garbage.

RUNNABLE JAVA PROGRAM

```
import java.util.Arrays;

public class DeleteAtSlot {
    public static void main(String[] args) {
        String[] bays = {"A", "B", "C", "D"};
        int used = 4;

        for (int i = 1; i < used - 1; i++) {
            bays[i] = bays[i + 1];
        }
        bays[used - 1] = null;
        used--;

        System.out.println(Arrays.toString(bays));
        System.out.println("used slots: " + used);
    }
}
```

OUTPUT

```
[A, C, D, null]
used slots: 3
```

The array still has four slots. "Size" is a number you maintain, and that bookkeeping is exactly what ArrayList does for you next week.

REVERSE

Reversing in place walks two pointers toward each other

Swap the ends, step inward, stop when they meet. The swap is the same three lines you wrote in the practice.

RUNNABLE JAVA PROGRAM

```
import java.util.Arrays;

public class ReverseInPlace {
    public static void main(String[] args) {
        int[] readings = {10, 20, 30, 40, 50};

        int left = 0;
        int right = readings.length - 1;

        while (left < right) {
            int held = readings[left];
            readings[left] = readings[right];
            readings[right] = held;
            left++;
            right--;
        }

        System.out.println(Arrays.toString(readings));
    }
}
```

OUTPUT

```
[50, 40, 30, 20, 10]
```

The loop stops when the pointers meet. On an odd length the middle element is left alone, which is exactly right.

KNOWLEDGE CHECK

The code is correct Java and the answer is still wrong

Nothing here crashes. Work out what the loop actually compares.

THINK IT THROUGH FIRST

WHAT DOES THIS PRINT?

```
double[] weights = {4.0, 9.0, 2.0};  
double smallest = 0.0;  
  
for (double w : weights) {  
    if (w < smallest) smallest = w;  
}  
  
System.out.println(smallest);
```

A. 2.0

B. 0.0

C. 4.0

D. It does not compile

Nothing is lighter than the seed, so the seed is the answer

Every weight is positive, so `w < 0.0` is never true and the if body never runs.

ANSWER

WHAT DOES THIS PRINT?

```
double[] weights = {4.0, 9.0, 2.0};
double smallest = 0.0;

for (double w : weights) {
    if (w < smallest) smallest = w;
}

System.out.println(smallest);
```

A. 2.0

B. 0.0

C. 4.0

D. It does not compile

WHY

Seeding a *minimum* at zero only works when negative values are possible. Seed from `weights[0]` instead and the answer is 2.0. The mirror-image bug seeds a *maximum* at zero, which happens to work for positive data — which is worse, because it hides until someone stores a negative number.

SECTION SIX

Two dimensions, and the limits of arrays

A grid is an array of arrays. And after that, what arrays genuinely cannot do.

A 2D array is an array whose slots hold arrays

`dock.length` is the number of rows. `dock[0].length` is the width of row zero.

RUNNABLE JAVA PROGRAM

```
public class DockGrid {  
    public static void main(String[] args) {  
        double[][] dock = new double[2][3];  
  
        dock[0][0] = 8.0;  
        dock[1][2] = 25.0;  
  
        System.out.println("rows: " + dock.length);  
        System.out.println("columns in row 0: " + dock[0].length);  
        System.out.println("dock[1][2] = " + dock[1][2]);  
    }  
}
```

OUTPUT

```
rows: 2  
columns in row 0: 3  
dock[1][2] = 25.0
```

Row first, then column. `dock[1][2]` is row 1, column 2 — not the other way round.

Walking a grid is one loop per dimension

The outer loop picks a row; the inner loop runs the full width of *that* row. Use `dock[row].length`, not a constant.

RUNNABLE JAVA PROGRAM

```
public class WalkTheDock {  
    public static void main(String[] args) {  
        double[][] dock = {{8.0, 25.0, 0.5}, {12.0, 3.0, 6.0}};  
  
        for (int row = 0; row < dock.length; row++) {  
            for (int col = 0; col < dock[row].length; col++) {  
                System.out.println("bay " + row + " slot " + col  
                                   + ": " + dock[row][col]);  
            }  
        }  
    }  
}
```

OUTPUT

```
bay 0 slot 0: 8.0  
bay 0 slot 1: 25.0  
bay 0 slot 2: 0.5  
bay 1 slot 0: 12.0  
bay 1 slot 1: 3.0  
bay 1 slot 2: 6.0
```

Rows can have different lengths in Java. `dock[row].length` is the only safe boundary.

SHALLOW COPY

Copying a 2D array copies the rows' references, not the rows

Fact two again, one level down. `Arrays.copyOf` gives you a new outer array pointing at the *same* inner arrays.

BROKEN PROGRAM

```
import java.util.Arrays;

public class ShallowCopy {
    public static void main(String[] args) {
        int[][] grid = {{1, 2}, {3, 4}};
        int[][] copy = Arrays.copyOf(grid, grid.length);

        copy[0][0] = 99;

        System.out.println("grid row 0: " + Arrays.toString(grid[0]));
        System.out.println("copy row 0: " + Arrays.toString(copy[0]));
    }
}
```

OUTPUT

```
grid row 0: [99, 2]
copy row 0: [99, 2]
```

To copy a grid for real, loop the rows and copyOf each one.

What arrays cannot do is the reason next week exists

Everything below is a real constraint, not a gap in your knowledge. Each one is solved by a specific tool you are about to meet.

THE LIMIT	WHAT IT FORCES YOU TO DO NOW	WHAT SOLVES IT
The length is fixed	Track a used counter and copy into a bigger array	ArrayList — Lesson 303.9
Related data lives in parallel arrays	Keep several arrays in step by hand, and never sort one alone	A class with fields — Lesson 303.7
Lookup by key means scanning	Linear search every time, or sort first	HashMap — Lesson 303.9
No built-in behaviour	Write total, search and reverse yourself	Collections and streams — Lessons 303.9 and 303.11

You are learning arrays because every one of those tools is built on top of one.

An array is the shape of almost every payload you will ever receive

The word changes — list, rows, records, results — but the shape is this one.

Where it arrives

A JSON response is an array of records. A database query returns rows. A CSV upload is lines. A Kafka poll returns a batch. In every case your first move is the loop you wrote on slide 12.

Why the reference rule matters at work

Handing an array to a method that mutates it is how one service quietly corrupts another service's data. "Did that call change my list?" is a real code-review question, and fact two is the answer.

What carries forward

- The **parallel-array problem** is the argument for classes, and you meet it head-on in the next lesson.
- **Insert and delete by shifting** is what `ArrayList` does internally — you will have written it by hand first.
- **Sorted plus binary search** is the idea behind every database index you will ever tune.
- **Seeding a running best** from the first element is the same bug in every language, forever.

Arrays are not a beginner topic you leave behind. They are the floor everything else is standing on.

SECTION SEVEN

Guided build: the heaviest-package report

One ticket that needs a copy, a sort, two searches and a total — and a decision behind each.

THE TICKET

Read the ticket, and notice what it does not say

It never mentions sorting, copying or searching. Those are your decisions, and criterion 4 quietly constrains all of them.

NP-121 Heaviest-package report

3 pts

Ops keeps two arrays that line up by position: the tracking numbers and the weights. They need a shift-end report, and the arrays must be usable afterwards exactly as they arrived.

ACCEPTANCE CRITERIA

1. Show every weight in ascending order
2. Report the heaviest weight and which tracking number owns it
3. Look up a given tracking number and report its weight
4. Leave the two original arrays in their original order
5. Report the total weight of the shelf

Criterion 4 is the one that decides the design. Read it before you write anything.

Five criteria, and criterion 4 rules out the obvious approach

The instinct is to sort the weights and read the last one. That satisfies criteria 1 and 2 and breaks criteria 3 and 4 at the same time.

- 1 Copy the weights, because we must sort without disturbing the original.
- 2 Sort the copy and take its last slot — that is the heaviest weight.
- 3 Search the *original* array for that weight to recover its slot number.
- 4 Use that slot in the tracking array — the pairing is still intact.
- 5 Search the tracking array by name for the lookup, and total the weights.

Why step 3 uses the original

The sorted copy has lost the pairing with the tracking numbers. Only the original array still lines up, so the slot number has to come from there.

Fact two, doing useful work

Everything here follows from knowing that `Arrays.sort` mutates and `=` does not copy. Without those two facts this design looks arbitrary.

The plan is five needs. Not one line of it names a Java method yet.

Step 1: rank the weights without disturbing them

Criteria 1, 2 and 4 at once. The decision is where to sort, not how.

STEP 1 OF 4 · COPY, THEN SORT

WHAT WE NEED Ascending order and the largest value, with the original order preserved.

OPTIONS `Arrays.sort(weights)` directly; sort a copy; or write a running-maximum loop

WE PICK **`Arrays.copyOf` then `Arrays.sort` on the copy**

BECAUSE Sorting in place satisfies criteria 1 and 2 but violates criterion 4 and destroys the pairing criterion 3 depends on. A running maximum would give us the heaviest but not the ascending list criterion 1 asks for. The copy gives us both.

STEP 1

```
double[] ranked = Arrays.copyOf(weights, weights.length);
Arrays.sort(ranked);
double heaviest = ranked[ranked.length - 1];
```

One extra line of copying is what keeps the other three criteria satisfiable.

Step 2: recover the slot, then the tracking number

We have the heaviest *weight*. Criterion 2 wants the tracking number, and only the original array can supply it.

STEP 2 OF 4 · SEARCH THE ORIGINAL

WHAT WE NEED Turn a weight back into the tracking number that owns it.

OPTIONS `Arrays.binarySearch` on the sorted copy; a linear search of the original; or keep a second sorted array of names

WE PICK **A linear search of the original weights array**

BECAUSE `binarySearch` would find the value in the *ranked* array, and that index means nothing to the tracking array. The slot we need is the position in the unsorted original, and only a linear scan can give us that.

STEP 2

```
int heaviestSlot = -1;
for (int i = 0; i < weights.length; i++) {
    if (weights[i] == heaviest) {
        heaviestSlot = i;
        break;
    }
}
```

An index is only meaningful in the array it came from. Carrying one across a sort is the classic parallel-array bug.

Step 3: both searches are the same shape, so name them once

Two loops that differ only in what they compare. Lifting them out of main is the first honest reason you have had to write a method.

STEP 3 OF 4 · LIFT THE SEARCHES OUT

ADD THESE INSIDE THE SAME CLASS, BELOW MAIN

```
public static int slotOfWeight(double[] values, double wanted) {  
    for (int i = 0; i < values.length; i++) {  
        if (values[i] == wanted) {  
            return i;  
        }  
    }  
    return -1;  
}  
  
public static int slotOfName(String[] names, String wanted) {  
    for (int i = 0; i < names.length; i++) {  
        if (names[i].equals(wanted)) {  
            return i;  
        }  
    }  
    return -1;  
}
```

return i replaces the break: leaving the method ends the loop too. **return -1** at the bottom means "scanned everything, not there."

Step 4: main, wired together

One copy, one sort, two named searches, one accumulator — and the originals untouched.

RUNNABLE JAVA PROGRAM

```
import java.util.Arrays;

public class HeaviestReport {
    public static void main(String[] args) {
        String[] tracking = {"NP-1001", "NP-1002", "NP-1003", "NP-1004", "NP-1005"};
        double[] weights = {8.0, 25.0, 0.5, 12.0, 3.0};

        double[] ranked = Arrays.copyOf(weights, weights.length);
        Arrays.sort(ranked);
        double heaviest = ranked[ranked.length - 1];

        int heaviestSlot = slotOfWeight(weights, heaviest);
        int wantedSlot = slotOfName(tracking, "NP-1004");

        double total = 0.0;
        for (int i = 0; i < weights.length; i++) {
            total = total + weights[i];
        }

        System.out.println("Ranked weights: " + Arrays.toString(ranked));
        System.out.println("Original order: " + Arrays.toString(weights));
        System.out.println("Heaviest: " + heaviest + " lbs");
        System.out.println("Owned by: " + tracking[heaviestSlot]);
        System.out.println("Lookup NP-1004: " + weights[wantedSlot] + " lbs");
        System.out.println("Total: " + total + " lbs");
    }
}
```

The originals came out exactly as they went in

That line of output is criterion 4 being satisfied, and it is the only evidence that the copy on line 7 was necessary.

OUTPUT

```
Ranked weights: [0.5, 3.0, 8.0, 12.0, 25.0]
Original order: [8.0, 25.0, 0.5, 12.0, 3.0]
Heaviest: 25.0 lbs
Owned by: NP-1002
Lookup NP-1004: 12.0 lbs
Total: 48.5 lbs
```

- ARRAYS.COPYOF** because sort mutates, and criterion 4 says the original must survive
- LINEAR, NOT BINARY** because the slot number has to be valid in the array that pairs with tracking
- .EQUALS FOR NAMES** because == on Strings compares references, the same rule as arrays
- BREAK IN BOTH SEARCHES** because there is nothing left to learn after the first match

What you can defend after today

Fixed length

insert and delete are shifting, and "size" is a counter you maintain

A reference

= aliases, == compares identity, methods can rewrite your data, copies are shallow

Zero-based

the last index is length - 1, and you never type the size as a number

The patterns

aggregate, linear search, binary search on sorted data, swap, shift, reverse

Next: the parallel-array problem gets solved properly — one class, one package, its own fields.